Amrita Vishwa Vidyapeetham

Real-Time Multi-Core Sorting and Median Filter Service

26ELC331 — Real-Time Operating Systems

Anmol Govindarajapuram Krishnan - CB.EN.U4EEE23103
Mauli Rajguru - CB.EN.U4EEE23120
Sanjay TS - CB.EN.U4EEE23136

October 17, 2026

Introduction
System Architecture
Implementation
Dataset & File Format
Dedicated Module Testbenches
Two-Fold Verification & Results
Applications & Conclusion

PART 1

Introduction

d.4 What the Service Does in One Breath

A **client** reads a continuous stream of floating-point sensor numbers, packs them into **64 KiB crates** (16 384 numbers each), and hands each crate to a **server** over QNX message passing.

For every crate, the server does two jobs:

- **Parallel Merge Sort**: Sorts a copy of the 16 384 numbers across worker threads to find the exact block median and range.
- **Rolling Median Filter**: Slides a small odd-width window across the time-ordered numbers to strip out one-off sensor spikes.

When done, the server sends a **QNX pulse** so the client never has to sit waiting.

Quick Vocabulary

- **Sample**: One `float` reading (4 B), e.g. `3.71 V`.
- **Record**: One moment holding all 4 channels (16 B).
- **Frame**: 128 samples returned per sensor read.
- **Block**: 16 384 samples = 64 KiB sent at once.

Course Rules Met

QNX synchronous `MsgSendv`, $\geq$ 64 KiB payload per transfer, multi-threaded sort, and async pulse notification.

d.5 Why a Median Filter Instead of an Average?

Suppose a battery sensor sits at 5.0 V, and one loose-wire glitch makes a single reading jump to 50.0 V:

5 Readings in Window	Average	Median	Real Value
5.0, 5.0, 5.0, 5.0, 5.0	5.0 V	5.0 V	5.0 V
5.0, 5.0, 5.0, 5.0, 50.0	**14.0 V**	**5.0 V**	5.0 V

A moving average gets pulled from 5 V up to **14 V** by one bad reading. The **median** sorts the 5 numbers and picks the middle one — still **5.0 V**.

- **Why an odd window width?** With 3, 5, or 7 samples, there is always one exact middle sample after sorting the window.
- **Why keep a separate copy?** Sorting the whole 16 384-sample block destroys time order. The rolling median filter must run on the original time-ordered sequence.
- **Why do both?** Full-block sort gives overall percentiles (min, median, max); the rolling filter gives a clean, spike-free waveform.

How the Modules Fit Together

Input Pipeline & IPC Layer

- `protocol.h` & `shm_region.c`: Define the 64 KiB block, shipping label, FNV-1a seal, IPC messages, and `/sortmed-results` shared memory.
- `sensor.c` & `block_loader.c`: Stream the binary recording through a 4096-record window and pack 16 384-sample crates.
- `client.c` & `server.c`: Send blocks via `MsgSendv`, notify via pulse, and read results from shared memory.

Sorting & Filtering Engines

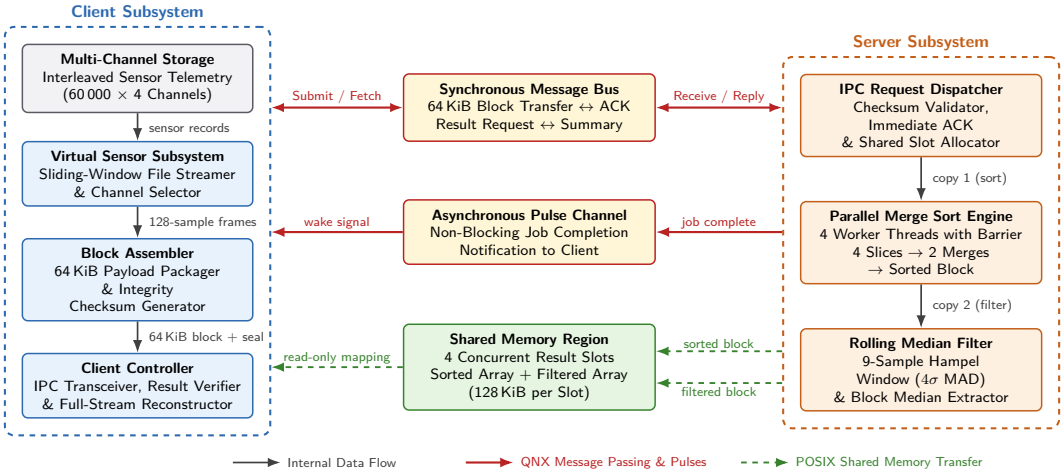
- `mergesort.c`: Splits each 16 384-sample block across POSIX worker threads to sort and extract the median.
- `median.c`: Runs a Hampel rolling median filter across the time-ordered copy to strip impulse spikes while leaving clean curves untouched.

`One copy is sorted for percentiles; a separate copy is filtered in time order`

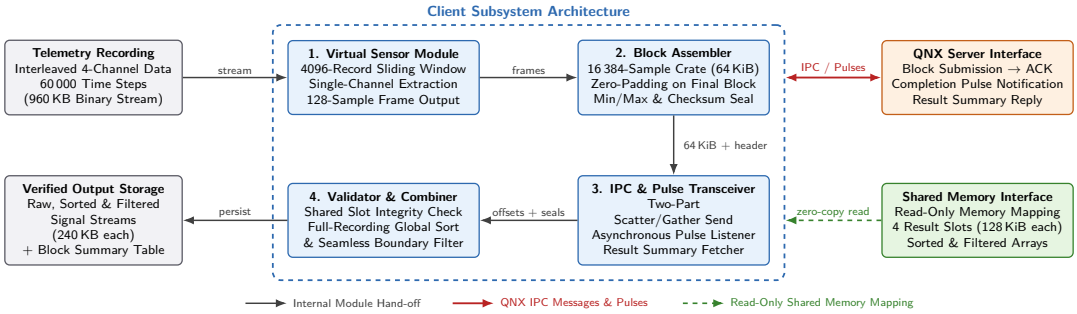
PART 2

System Architecture

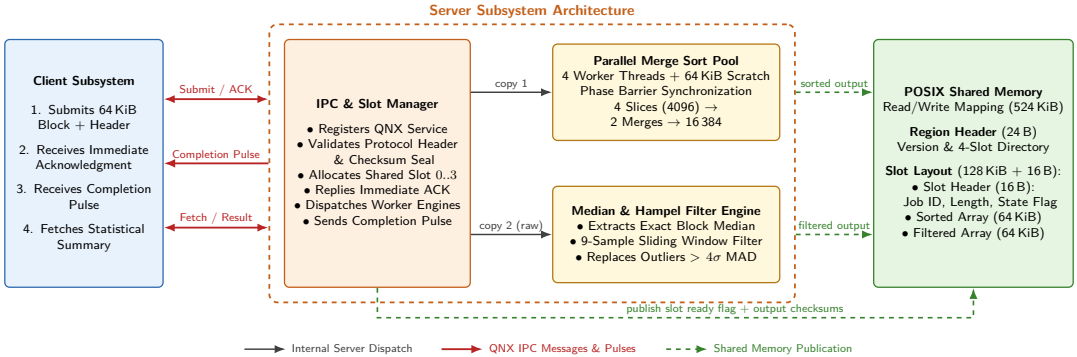
d.8 System Architecture: Block Diagram



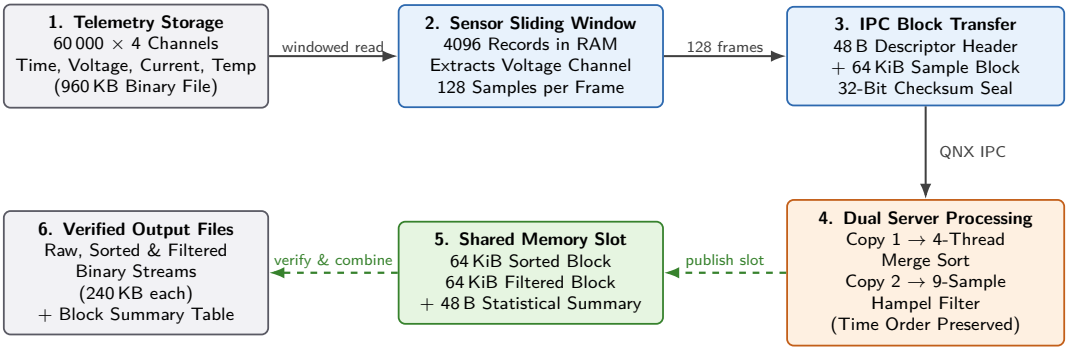
d.9 Client Architecture: Block Diagram



d.10 Server Architecture: Block Diagram



d.11 Data Flow: Block Diagram



PART 3

Implementation

`sensor.c` wraps the binary recording on disk and makes it behave like live hardware:

- **Checks file on open**: Verifies byte size is a multiple of 4 floats $\times$ 4 B = 16 B. Rejects bad rates, zero strides, or invalid offsets right away.
- **4096-record sliding window**: Keeps only 4096 records in RAM and refills via `fread()` when the cursor walks past the window.
- **128-sample frames**: Like a hardware ADC FIFO, `sensor_read()` returns at most 128 samples per call.
- **Paced or unpaced**: `sensor_read_paced()` sleeps against `CLOCK_MONOTONIC` to match real-time Hz; unpaced reads run at full CPU speed.

Picking Voltage from 4 Channels
Each record stores `[Time, Voltage, Current, Temp]`. To read only Voltage, we open with `stride = 4, offset = 1`: step by 4 floats, always pick slot 1.

Drift-Free Sensor Timestamp
Computed directly from the sample counter: $t_{ns} = (\text{delivered} \times 10^9)/\text{rate_hz}$. At 100 Hz, sample 100 is always 1.000 s, unaffected by OS sleep jitter.

d.14 Virtual Sensor: Line-by-Line Code Walkthrough

```
/* Inside sensor_read() in app/sensor.c */
while (n < cap) {
    if (s->window_count == 0 ||
        s->next < s->window_first ||
        s->next >= s->window_first + s->window_count) {
        if (sensor_fill(s, s->next) != 0)
            return (n > 0) ? (int)n : -1;
    }
    size_t off = (s->next - s->window_first) *
                 s->cfg.stride + s->cfg.offset;
    dst[n++] = s->window[off];
    s->next++;
    s->delivered++;
}
```

cap is capped at one 128-sample frame per read call.

- **Lines 3–5 (Window check)**: Is our next sample index `s->next` outside the 4096 records currently in RAM?
- **Lines 6–7 (`sensor_fill`)**: If outside, seek the file to `s->next` and `fread()` the next 4096-record chunk into `s->window`.
- **Lines 9–10 (Pick channel)**: With `stride=4, offset=1`, sample $0, 1, 2 \ldots$ reads window index $1, 5, 9 \ldots$ (only the Voltage floats).
- **Lines 11–13 (Deliver & advance)**: Copy float into `dst`, advance file cursor `s->next`, and bump `s->delivered` (clock counter).

d.15 How the Block Loader Works (app/block_loader.c)

The sensor hands out small 128-sample frames, while the server expects full **16 384-sample (64 KiB) blocks**. `block_loader.c` bridges the two:

- **Single place for channel offsets**: `block_loader_channel_offset()` maps channel names to their slot in the 4-float record:

  `VOLTAGE` → 1, `CURRENT` → 2, `TEMP` → 3

- **Gathers frames into a crate**: Loops over `sensor_read()` until 16 384 samples are collected (128 frames) or the file ends.
- **Fills out the shipping label**: Scans the collected samples for `min_val` and `max_val`, sets `seq`, and stamps the FNV-1a checksum.

What happens on the 4th block?
60 000 samples $= 3 \times 16\,384 + \mathbf{10\,848}$.
The 4th block has 10 848 real readings. The loader zeroes the remaining **5 536 slots** so leftover RAM garbage never crosses processes.

Why checksum only `desc->samples`?
The seal hashes only the **10 848 real samples**, not the 5 536 padding zeros, so the server sorts and medians only real battery readings.

d.16 Block Loader: Line-by-Line Code Walkthrough

```
/* Inside block_loader_next() in block_loader.c */
got = sensor_read_block(l->sensor,
                        block->samples,
                        PROTO_BLOCK_SAMPLES);
if (got <= 0) return got;

if ((uint32_t)got < PROTO_BLOCK_SAMPLES) {
    memset(&block->samples[got], 0,
           (PROTO_BLOCK_SAMPLES - got) * sizeof(float));
}
desc->samples  = (uint32_t)got;
desc->checksum = proto_checksum_block(block->samples,
                                      desc->samples);
```

`PROTO_BLOCK_SAMPLES` = 16 384. Produces a clean 64 KiB crate + label.

- **Lines 2–4 (`sensor_read_block`)**: Pulls 128-sample frames until `block->samples` holds 16 384 floats (or hits EOF). Returns count `got`.
- **Line 5 (EOF / Error check)**: Returns 0 when the recording finishes, or < 0 on read error.
- **Lines 7–10 (Zero-pad short block)**: When `got` is 10 848 on the last crate, `memset` zeroes indices 10 848..16 383 so all 65 536 B are clean.
- **Lines 11–13 (Record count & seal)**: Stores `got` in `desc->samples` and hashes only those `got` floats into `desc->checksum`.

app/protocol.h defines the 5 messages used across QNX IPC:

Step / Message	Size	What Happens
1. SUBMIT + BLOCK	48 B + 64 KiB	Client sends label + crate (2-part IOV)
2. ACK	8 B	Server checks seal, replies right away
3. PULSE (code 16)	1 B code	Server taps client when sort finishes
4. FETCH	8 B	Client asks for finished job's numbers
5. RESULT	48 B	Server returns median, min/max, offsets

Why reply with ACK before sorting?
In QNX, MsgSendv blocks the client until the server calls MsgReply. Replying with ACK immediately lets the client keep reading the sensor while workers sort.

Compile-Time Safety Guards
STATIC_ASSERT pins sizeof(proto_block_t) to 65 536 B. Message IDs start at 0x1000, well above QNX system messages (0x000–0x1FF).

d.18 Protocol Definition: Shared Results Region

The pulse only says *a job is done*; the arrays are handed over in a **named shared region**, defined in the same header:

Part	Size	Contents
region header	24 B	magic, version, slot count
slot header	16 B	job id, state word
sorted[]	64 KiB	the sorted block
filtered[]	64 KiB	the time-ordered copy

Region `/sortmed-results`: one header plus **4** slots, so four results can be in flight.

A state word, not a lock
The server publishes a slot by writing `WR2D` to its state word. A slot still reading `FREE` has not been written. The client re-reads that one word — no mutex is shared across the programs.

The result points, it does not carry
`proto_result_t` holds `sorted_offset`, `filtered_offset`, `length`, and two checksums, all 32-bit. Offset 0 means "not produced".

Three tests pin the region: its name, its layout, and that the result describes it.

d.19 How Parallel Merge Sort Splits & Merges (mergesort.c)

How a 16 384-sample crate is sorted across **4 worker threads**:

- **How many slices?**: $k = \min(n/1024, \text{threads}) \rightarrow$ **4 slices** of **4 096 samples** each. (Blocks < 4096 run on 1 thread.)
- **Phase 0 — Sort 4 slices (4 tasks)**: All 4 workers run in parallel, sorting their 4 096-sample slice into scratch.
- **Phase 1 — First merge (2 tasks)**: Two workers merge pairs of 4 096 into two runs of **8 192** in the main buffer.
- **Phase 2 — Final merge (1 task)**: One worker merges the two 8 192 runs into the final **16 384** sorted array.

Total plan: **4 partitions, 2 merge phases, 7 tasks** $(4 + 2 + 1)$.

Ping-Pong Scratch Buffer
One 64 KiB scratch buffer is allocated when the pool starts. Phases bounce between `buf_a` and `buf_b` with zero per-job `malloc`.

Thread-Invariant Output
When merging two halves, ties always pick from the left half (`a[i] <= b[j]`). Output is **bit-for-bit identical** across 1, 2, 4, and 8 threads.

d.20 Parallel Merge Sort: Line-by-Line Code Walkthrough

```
/* 1. Manager posts a phase & waits (mergesort.c) */
static void post_phase(mergesort_pool_t *p, unsigned ntasks) {
    pthread_mutex_lock(&p->lock);
    p->ntasks = ntasks; p->next_task = 0; p->tasks_done = 0;
    pthread_cond_broadcast(&p->work_ready);
    while (p->tasks_done < p->ntasks)
        pthread_cond_wait(&p->phase_done, &p->lock);
    pthread_mutex_unlock(&p->lock);
}
/* 2. Inside each worker thread loop */
unsigned idx = p->next_task++;
mergesort_task_t t = p->tasks[idx];
pthread_mutex_unlock(&p->lock);
run_task(&t); /* sorts/merges unlocked in parallel */
pthread_mutex_lock(&p->lock);
if (++p->tasks_done == p->ntasks)
    pthread_cond_signal(&p->phase_done);
```

- **Lines 3–5 (Publish phase)**: Locks mutex, sets `ntasks` (4, then 2, then 1), resets counters, and broadcasts `work_ready`.
- **Lines 6–7 (Phase barrier)**: Manager sleeps on `phase_done` until all tasks in this phase finish.
- **Lines 11–14 (Parallel work)**: Worker claims ticket `next_task++`, **unlocks mutex immediately**, and runs `run_task()` on its core.
- **Lines 15–17 (Wake manager)**: Last finisher (`tasks_done == ntasks`) signals `phase_done` to advance phases.

`median.c` computes both the sorted-array median and a **Hampel rolling median filter** on the time-ordered signal:

- **Centered odd window**: Slides an odd window (default `width = 9`, range 3..31) across the original samples.
- **Window median & MAD**: Orders the small window on the stack via insertion sort to get the local `median` and the median absolute deviation (`mad`).
- **Replaces only true spikes**: Converts MAD to robust sigma ($\sigma = 1.4826 \times \text{MAD}$). Replaces a sample **only** if it exceeds 4σ from the window median; clean samples pass through untouched.

Why Hampel Instead of Plain Median?
A plain median replaces every point and rounds off clean voltage ramps. Hampel replaces only the 10–40σ spikes and leaves clean readings bit-exact.

Seamless Block Boundaries
Accepts up to `(width-1)/2` neighbour samples (`left`, `right`) so block seams produce the exact same output as filtering the whole recording at once.

d.22 Rolling Median Filter: Line-by-Line Code Walkthrough

```
/* Inside median_filter_block() in app/median.c */
for (k = 0; k < count; k++)
    window[k] = sample_at(lo + (ptrdiff_t)k, left,
                          (ptrdiff_t)nleft, cur, (ptrdiff_t)n,
      right);
sort_small(window, count);
median = middle_of(window, count);

for (k = 0; k < count; k++)
    deviation[k] = fabsf(window[k] - median);
sort_small(deviation, count);
mad = middle_of(deviation, count);

if (fabsf(centre - median) >
    cfg->threshold * MEDIAN_MAD_TO_SIGMA * mad) {
    out[i] = median;
    replaced++;
} else {
    out[i] = centre;
}
```

- **Lines 2–6 (Window median)**: Copies the 9-sample neighborhood around `cur[i]` into `window[]`, insertion-sorts it (`sort_small`), and picks the middle value.
- **Lines 8–11 (Compute MAD)**: Measures each neighbor's distance `|x - median|`, sorts those distances, and takes their middle value (`mad`).
- **Lines 13–18 (Spike test)**: If `centre` is further than `4.0 * 1.4826 * mad`, replaces it with `median` and bumps `replaced`; otherwise keeps `centre`.

How the QNX Server & Client Work (server.c, client.c)

Server (`sortmed-server`):

- Creates `/sortmed-results` shared memory (`shm_region_create`) and attaches channel `"sortmed"`.
- Receives `{submit, block}` via `MsgReceivev`, verifies checksum, and replies `ACK` immediately.
- Sorts one copy in shared memory via `ms_sort()`, filters a second copy via `median_filter_block()`, marks slot `WR2D`, and sends pulse code 16.

Client (`sortmed-client`):

- Attaches `"sortmed-client"` for pulses and maps `/sortmed-results` read-only.
- Sends each 64 KiB block with `MsgSendv`, waits for the completion pulse, and sends `FETCH`.
- Verifies slot state `WR2D`, bounds, and FNV-1a checksums, then writes `raw.bin`, `sorted.bin`, `filtered.bin`, and `medians.csv`.

Server Job Processing: Line-by-Line Code Walkthrough

```
/* Inside process_submit() in app/server.c */
slot->state = PROTO_SLOT_FREE;
memcpy(sorted, block->samples, n * sizeof(float));
if (ms_sort(pool, sorted, n) != 0) return -1;

median_config_default(&cfg);
if (median_filter_block(&cfg, NULL, 0, block->samples, n,
                        NULL, 0, filtered, &outliers) != 0)
    return -1;

job->result.median = median_of_sorted(sorted, n);
job->result.outliers_removed = outliers;
job->result.sorted_checksum =
    proto_checksum(sorted, n * sizeof(float));
job->result.filtered_checksum =
    proto_checksum(filtered, n * sizeof(float));
slot->state = PROTO_SLOT_WRITTEN; /* 'WR2D' */
```

- **Lines 2–4 (Sort copy in shm)**: Marks slot `FREE`, copies raw samples into the slot's `sorted[]` array, and runs the 4-worker parallel merge sort.
- **Lines 6–9 (Filter time-ordered copy)**: Runs the width-9 Hampel filter on the untouched `block->samples` into the slot's `filtered[]` array.
- **Lines 11–17 (Seal & publish)**: Records block median, outlier count, and both output checksums, then flips `slot->state` to `WR2D` right before pulsing the client.

PART 4

Dataset & File Format

Synthetic Battery Telemetry (PyBaMM)

Why generate our own battery dataset instead of random numbers?

- **Real electrochemical curve**: Simulated an LG M50 lithium-ion cell (`Chen2020`) discharging at 0.5C, 1C, and 2C.
- **4 channels at 100 Hz**: 60 000 time steps (10 minutes) of Time, Voltage, Current, and Temperature.
- **Realistic sensor faults**: Added background sensor hiss (AWGN) plus **1.5% sharp impulse spikes** (10–40σ), bursts, and dropouts.
- **Saved the answer key**: Kept the clean curve and exact spike mask offline to score filter accuracy.

Dataset Property	Value
Discharge runs	3 (0.5C, 1C, 2C)
Moments per run	60 000 (at 100 Hz)
Channels per moment	4 (t, V, I, T)
Total float readings	720 000
1C production file	960 000 B (4 blocks)
Injected spike cells	8 040 (1.489%)

Gives us a repeatable, realistic signal with known medians and known spikes.

d.27 One-Time Clarification: Planar vs Interleaved Files

Planar File (Single-Channel Check)
One file per signal (`r1_voltage.bin`). 60 000 voltages in a row (240 000 B): `[V0, V1, V2...]` (`stride=1, offset=0`).

Interleaved File (Production Payload)
One unified file (`Chen2020_c1...bin`). 60 000 moments × 4 channels (960 000 B): `[t0, V0, I0, T0, t1, V1, I1, T1...]` (`stride=4, offset=k`).

Why This Distinction Matters in Code
If you sort an interleaved file directly without picking channel offset k, you sort **seconds against volts against amps against kelvin** — the numbers sort, but the result is meaningless!

- `block_loader_channel_offset()` ensures we extract only one physical quantity (Voltage at `offset = 1`).
- Planar files were used **only once in tests** to verify that strided extraction produces the exact same 4 checksums.
- **All production runs use the Interleaved file.**

File: `Chen2020_c1_T25_discharge.bin` (960 000 B)

Every 10 ms sample instant is stored as a 16-byte record of 4 floats:

Offset k	Bytes	Channel	Unit
0	0–3	Time	Seconds (s)
1	4–7	**Voltage**	Volts (V)
2	8–11	Current	Amperes (A)
3	12–15	Temperature	Kelvin (K)

60 000 records → **4 voltage blocks**: 3 full crates (16 384) + 1 short crate (10 848).

How `stride=4, offset=1` Reads Voltage
Record i starts at float index $4i$. Adding `offset = 1` lands directly on `voltage_V` at float index $4i + 1$ (bytes $16i + 4$).

Expected Voltage Block Seals (FNV-1a)

- Block 0 (0..16383): `0xe90d24d2`
- Block 1 (16384..32767): `0xc98cd3b2`
- Block 2 (32768..49151): `0x14c728bb`
- Block 3 (49152..59999): `0x362ab592`

PART 5

Dedicated Module Testbenches

d.30 Testbench 1: Virtual Sensor (test_sensor)

1. What it tests

Proves `sensor.c` refills the 4096-record window, extracts channels with `stride=4`, caps 128-sample frames, computes timestamps, and rejects invalid configs.

2. How it is done

- Passes broken inputs (missing file, 3 B non-float file, zero rate, `offset >= stride`) to verify rejection.
- Streams 10 000 samples across multiple window refills against a known ramp $[0, 1, 2 \ldots]$.
- Checks strided interleaved reads against expected channel values.

3. Output

- Clear diagnostic printed for each bad input (e.g. `not a multiple of 4-byte float`).
- Bit-for-bit match across all window refills, loop wrap-arounds, and timestamps.
- **15 tests, 642 checks passed (0 failed, 0 skipped).**

`Confirms the sensor stream is rock-solid before packing blocks`

d.31 Testbench 2: Protocol Contract (test_protocol)

1. What it tests

Proves `protocol.h` locks down the 64 KiB block size, struct byte layouts, safe QNX message numbers, FNV-1a checksum verification, and the `/sortmed-results` shared memory layout.

2. How it is done

- Checks `sizeof(proto_block_t) == 65536` and message header sizes (48 B / 8 B).
- Builds a valid `SUBMIT` message, verifies it passes, then flips **1 bit** in sample 100 and re-runs `proto_submit_verify()`.
- Checks the 24 B header and 4-slot offsets of the shared results region.

3. Output

- 1-bit tampered payload is immediately caught and returns `PROTO_ERR_CHECKSUM (-5)`.
- Bad magic numbers, wrong versions, and out-of-range sample counts return their exact error codes.
- **12 tests, 137 checks passed (0 failed, 0 skipped)**.

`Guarantees client and server agree on every byte and offset`

d.32 Testbench 3: Block Loader (test_block_loader)

1. What it tests

Proves `block_loader.c` gathers 128-sample frames into 16 384-sample crates, zero-pads the short final crate, maps channel offsets, and stamps valid FNV-1a seals.

2. How it is done

- Feeds a 20 000-sample ramp and checks that Block 0 gets 16 384 samples and Block 1 gets 3 616 samples plus 12 768 zeros.
- Loads the 60 000-record interleaved battery file with `VOLTAGE` selected and verifies all 4 blocks via `proto_submit_verify()`.

3. Output

- Every sample across all blocks matches the source recording bit-for-bit, with zeroed tail slots confirmed.
- Produces the 4 expected voltage block seals (`0xe90d24d2`, `0xc98cd3b2`, `0x14c728bb`, `0x362ab592`).
- **8 tests, 116 checks passed (0 failed, 0 skipped).**

`Guarantees every 64 KiB crate arriving at the server is valid`

d.33 Testbench 4: Parallel Merge Sort (test_mergesort)

1. What it tests

Proves `mergesort.c` sorts accurately across multiple worker threads, synchronizes phases cleanly without race conditions, and gives identical output regardless of thread count.

2. How it is done

- Compares sorted output against standard C `qsort` on random arrays, reverse-sorted arrays, all-equal arrays, and ± 0.0 / denormal floats.
- Sorts the same 16 384-sample block with **1, 2, 4, and 8 threads** and compares every output byte.
- Sorts 100 blocks back-to-back on the same pool to stress-test barrier reuse.

3. Output

- 100% bit-identical sorted arrays across 1, 2, 4, and 8 worker threads.
- Phase plan verified on 16 384 samples with 4 threads: **4 partitions, 2 merge phases, 7 tasks**.
- **13 tests, 578 checks passed (0 failed, 0 skipped).**

`Guarantees fast multi-core sorting with zero thread bugs`

Testbench 5: Rolling Median & Shared Memory (test_median)

1. What it tests

Proves `median.c` strips impulse spikes without altering clean curves, matches reference output across block seams, and checks shared-memory bounds.

2. How it is done

- Compares `median_filter_block()` bit-for-bit against a brute-force `qsort` reference across widths 3..31.
- Filters 5000 samples across block seams with neighbour context to verify it matches whole-signal filtering.
- Tests `shm_range_valid()` against out-of-bounds offsets and `UINT64_MAX` overflow traps.

3. Output

- Clean signals remain untouched; injected spikes drop RMS error back to the 1σ noise floor.
- Block-by-block filtering with neighbour context is bit-identical to full-signal filtering.
- **12 median tests (7551 checks) + 13 shm checks passed (0 failed).**

`Confirms spike removal and safe shared-memory offset math`

PART 6

Two-Fold Verification & Results

d.36 Why We Tested in a Two-Fold Manner

We verified every module in two stages so logic bugs and hardware timing are tested separately:

Fold 1: Host PC Verification (Logic First)

- Compiled with `gcc -pthread` on the PC before touching hardware.
- Catches window boundary bugs, 1-bit checksum errors, block-seam filter bugs, integer overflows, and thread races right away.
- Runs all 6 testbenches (`test_sensor`, `test_protocol`, `test_block_loader`, `test_mergesort`, `test_median`, `test_shared_memory`).

61 tests, 9037 checks — 0 failed, 0 skipped.

Fold 2: On-Target Raspberry Pi 4 (QNX 8.0)

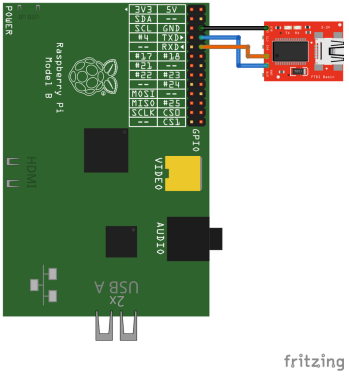
- Cross-compiled for ARM64 (`qcc -Vgcc_ntoaarch64le`) and packed into the QNX boot image (`ifs-rpi4.bin`).
- Runs the testbenches, `sortmed-server`, and `sortmed-client` on the 4 physical Cortex-A72 cores over UART serial.
- Proves real QNX message passing, pulse delivery, shared memory, and multi-core speedup on the board.

100% pass rate across all target tests and hardware demos.

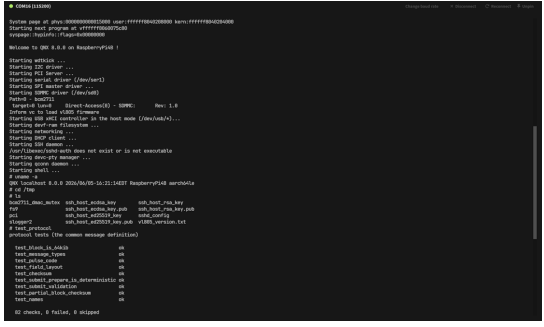
Target Testbench Setup

Hardware Target
Raspberry Pi 4 Model B with 4 Cortex-A72 cores running QNX 8.0.0. Booted directly from SD card into RAM IFS image.

Zero Network Dependency
Controlled strictly via FTDI USB-to-serial adapter on GPIO 14/15 (`/dev/ser1`) at 115200 8N1. Self-contained and noise-free.

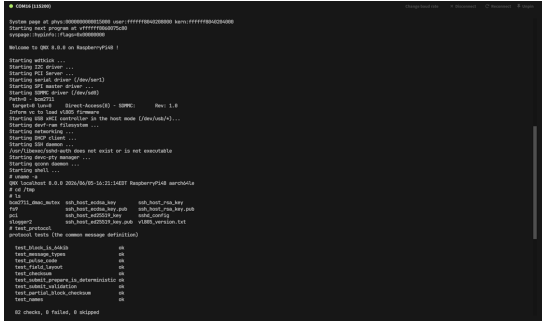


d.38 Console: Board Boot and Initialization



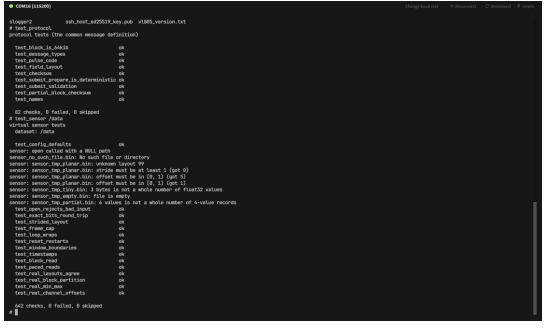
Raspberry Pi 4 boot sequence under QNX 8.0.0. All four Cortex-A72 cores are brought online by `procnto-smp-instr`. System shell initializes on serial console.

d.39 Console: Protocol Layer Verification



`test_protocol` executing on target. Pins the exact 64 KiB block size, application message ID ranges, pulse codes, FNV-1a checksum, and the shared results region.

d.40 Console: Virtual Sensor Verification



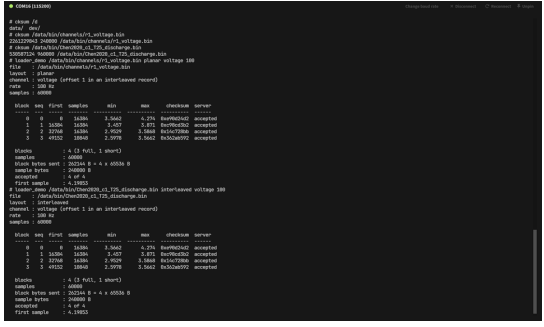
`test_sensor /data`: 642 checks passed. Console verifies robust error trapping: explicitly naming and rejecting invalid strides, bad offsets, and unaligned files.

d.41 Console: Block Loader Verification



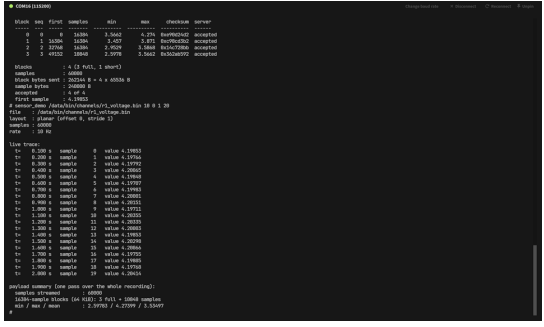
`test_block_loader /data`: 116 checks passed. Confirms strided record extraction produces deterministic 64 KiB blocks and detects bit corruption.

d.42 Console: On-Board Payload Verification



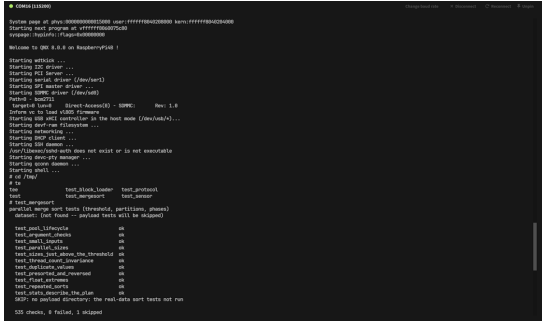
Direct `cksum` comparison of embedded IFS files, followed by `loader_demo` reading the interleaved file. Checksums match host dataset; blocks 0–3 accepted.

d.43 Console: Live Sensor Streaming Demo



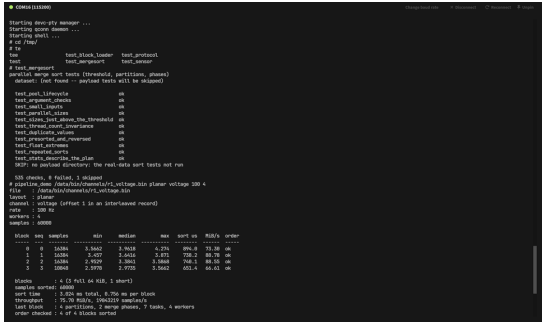
sensor_demo streaming at 10 Hz with hardware monotonic pacing. Reads sample-by-sample, aggregating 60 000 readings into 4 structured blocks.

d.44 Console: Parallel Sorting Verification



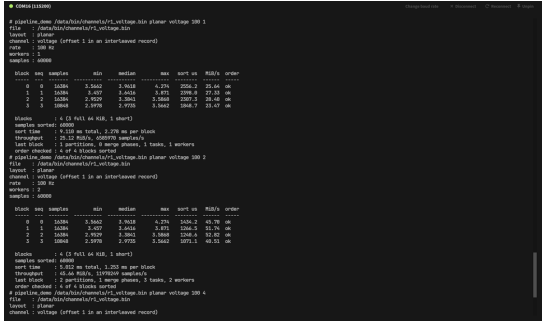
`test_mergesort`: 578 checks passed. Tests edge boundaries (0 to 64 samples), duplicate values, reverse sequences, and thread invariance.

d.45 Console: Real-Time Multi-Core Execution



`pipeline_demo` on real voltage telemetry with 4 workers. Phase plan dynamically reflects load: **4 partitions, 2 merge phases, 7 tasks**.

d.46 Console: Core Scaling Sweep



Worker sweep (1, 2, 4 threads) sorting 64 KiB blocks. Execution time drops from 2.278 ms $\rightarrow$ 1.253 ms $\rightarrow$ 0.757 ms. Medians and ranges remain perfectly identical.

Benchmark Analysis and Throughput

Workers	Time/Block	Throughput	Speed-up
1	2.278 ms	25.12 MiB/s	1.00×
2	1.253 ms	45.66 MiB/s	1.82×
4	**0.757 ms**	**75.55 MiB/s**	**3.01×**

Measured on physical Raspberry Pi 4 running QNX 8.0.0 SMP.

Massive Real-Time Headroom
One 64 KiB block represents **163.84 seconds** of 100 Hz sensor telemetry. Sorting completes in **0.757 ms** — over **216 000×** real-time speed.

- Clean multi-core scaling (3.01× speedup on 4 physical Cortex-A72 cores).
- Exact same medians (3.9618 V, 3.6416 V, 3.3841 V, 2.9735 V) across 1, 2, and 4 workers.

End-to-End QNX Service Execution on Raspberry Pi 4

`sortmed-server` & `sortmed-client` on `Chen2020_c1_T25_discharge.bin`:

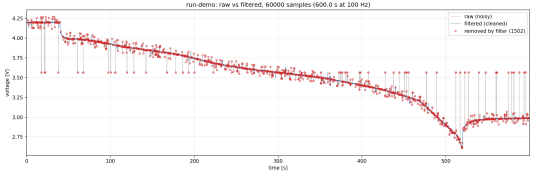
Block	Samples	Median (V)	Spikes	Server Time
0 (Job 1)	16 384	3.961808	431	7.51 ms
1 (Job 2)	16 384	3.641635	382	7.32 ms
2 (Job 3)	16 384	3.384104	406	7.30 ms
3 (Job 4)	10 848	2.973536	283	5.01 ms
Global	**60 000**	**3.566194**	**1 502**	**27.14 ms**

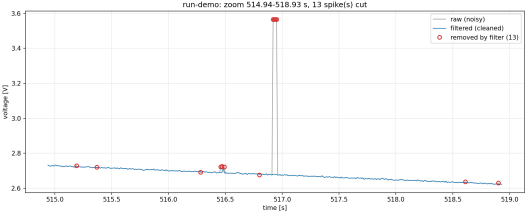
Time breakdown per 64 KiB block: **0.757 ms** 4-worker merge sort + ∼**6.5 ms** width-9 Hampel filter & FNV-1a checksums.

Round-Trip Bit-Exact Proof
`cksum raw.bin` matches the single-channel reference file (2261229843, 240 000 B), proving strided extraction and IPC reassembly lost zero bits.

Persisting Outputs on QNX
Since `/tmp` is RAM (`/dev/shmem`), the SD card FAT partition (`/dev/sd0t1`) is mounted at `/var/mnt` to save `raw.bin`, `sorted.bin`, `filtered.bin`, and `medians.csv`.

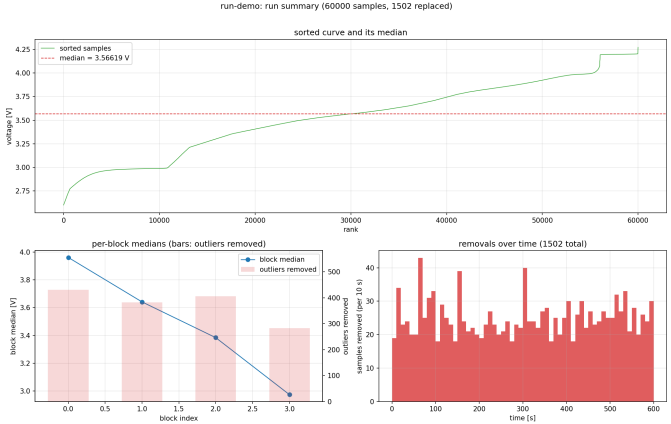
d.49 Hardware Output: Noisy vs Hampel-Filtered Telemetry





Left: Full 60 000-sample discharge curve (`raw.bin` vs `filtered.bin`) across the 4 block boundaries. **Right**: 2 000-sample zoom showing how the width-9 Hampel filter strips all **1 502 impulse spikes** while leaving the true discharge curve untouched.

d.50 Hardware Output: Sorted Distribution & Per-Block Summary



Global Sorted Curve
`sorted.bin` holds all 60 000 samples in monotonically increasing order with global median **3.5662 V**.

Per-Block Summary (`medians.csv`)
Tracks the voltage drop across blocks (3.962 V $\rightarrow$ 2.974 V) and logs the exact outlier count removed per block (431, 382, 406, 283).

PART 7

Applications & Conclusion

Practical Industrial Applications

- **Battery Management Systems (BMS)**: Electric vehicle battery packs suffer sharp switching spikes during inverter transients. Rolling median filtering strips the spikes while parallel sorting tracks cell voltage distributions.
- **Industrial Edge Gateways**: Factory controllers gather high-rate vibration and temperature feeds; handing 64 KiB blocks to worker threads keeps sensor polling loops from stalling.
- **Automotive Sensor Fusion (ADAS)**: Radar and wheel-speed streams need deterministic, spike-free filtering and fast percentile calculation on QNX safety controllers.
- **Flight & Railway Recorders**: Summarizing continuous multi-channel telemetry into checksum-verified statistical blocks.

Fits any embedded QNX system that needs spike removal and fast multi-core percentiles without blocking sensor reads.

Summary and Project Status

- **Input Pipeline**: 64 KiB block contract (`protocol.h`), sliding-window virtual sensor (`sensor.c`), strided block loader (`block_loader.c`), and PyBaMM interleaved dataset.
- **Parallel Sort & Rolling Filter**: Multi-threaded merge sort (`mergesort.c`) with POSIX phase barriers + Hampel rolling median filter (`median.c`).
- **QNX Client–Server Service**: `MsgSendv`/`MsgReceivev` 64 KiB transfer, `/sortmed-results` shared memory (`shm_region.c`), and pulse code 16 notification (`server.c`, `client.c`).
- **Two-Fold Verification**: 61 host tests (**9037 checks**, 0 failures) + full end-to-end QNX Raspberry Pi 4 verification.
- **Target Performance**: **0.757 ms** sort time (3.01× speedup) and ∼**7.3 ms** total server time per 64 KiB block across 4 Cortex-A72 cores.
- **Verified Outputs**: Strips **1 502 spikes** across 60 000 samples and persists bit-exact `raw.bin`, `sorted.bin`, `filtered.bin`, and `medians.csv`.

Questions and Discussion

THANK YOU

Real-Time Multi-Core Sorting and Median Filter Service

Anmol Govindarajapuram Krishnan - CB.EN.U4EEE23103
Mauli Rajguru - CB.EN.U4EEE23120
Sanjay TS - CB.EN.U4EEE23136

26ELC331 - Real-Time Operating Systems